

**Smt. Chandaben Mohanbhai Patel Institute of Computer Applications,
CHARUSAT
BCA Semester-III**

CAUE302 : Python Programming

Variables, Data Types, Input/Output and Operators

1. Introduction

- Students who have studied C, C++, or Java may notice that Python variables are declared differently.
- In C language:

```
int a = 2;
float b = 10.5;
char name[] = "Krupa";
```
- In Java:

```
int a = 2;
String name = "Krupa";
```

In both languages, the datatype must be specified before creating a variable.

However, in Python we simply write:

```
a = 2
name = "XYZ"
salary = 40000.50
```

Python automatically identifies the datatype based on the value assigned to the variable.

Dynamic Typing in Python

Python is a **Dynamically Typed Language**.

Definition

In Python, the datatype of a variable is determined automatically at runtime according to the value assigned to it.

Therefore, programmers do not need to explicitly declare datatypes while creating variables.

Examples

Integer Value

```
a = 10

print(a)
print(type(a))
```

Output

```
10
<class 'int'>
```

String Value

```
name = "Krupa"

print(name)
print(type(name))
```

Output

```
Krupa
<class 'str'>
```

Float Value

```
salary = 25000.50

print(salary)
print(type(salary))
```

Output

```
25000.5
<class 'float'>
```

Variable Can Change Its Datatype

Unlike C or Java, Python allows a variable to store different types of values during execution.

```
a = 10

print(a)
print(type(a))

a = "Hello"

print(a)
print(type(a))
```

Output

```
10
<class 'int'>

Hello
<class 'str'>
```

This demonstrates the dynamic typing feature of Python.

Static Typing vs Dynamic Typing

Static Typing	Dynamic Typing
Datatype must be declared	Datatype assigned automatically
Examples: C, C++, Java	Example: Python
Less flexible	More flexible
Type checked during compilation	Type determined during execution

2. Variables in Python

Introduction

- A variable is a named memory location used to store data in a program.
- Variables allow us to store information such as:
 - Names
 - Age
 - Marks
 - Salary
 - Address
- and use them whenever required in the program.

Syntax

```
variable_name = value
```

Example

```
name = "Mohan"
age = 23
salary = 25000.50
```

Here,

- name, age, and salary are variables.
- = is the assignment operator.
- Values are stored in the respective variables.

Example Program

```
name = "Mohan"
age = 23

print(name)
print(age)
```

Output

Mohan
23

Rules for Naming Variables

- **Valid Variable Names**

```
student_name = "Riya"  
age = 20  
marks1 = 85  
_total = 500
```

- **Invalid Variable Names**

```
1name = "Riya"
```

Reason:

Variable name cannot start with a digit.

```
student-name = "Riya"
```

Reason:

Hyphen (-) is not allowed.

```
class = "Python"
```

Reason:

Keywords cannot be used as variable names.

Here is a list of the Python keywords. Enter any keyword to get more help.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

help> _

Keywords in Python

Case Sensitivity

Python is case-sensitive.

Example:

```
name = "Krupa Pathak"
Name = "Xyz"
print(name)
print(Name)
```

Output

```
Krupa Pathak
Xyz
```

Here:

name

and

Name

are considered different variables.

Multiple Variable Assignment

Python allows assigning multiple values in a single statement.

```
a, b, c = 10, 20, 30
print(a)
print(b)
print(c)
```

Output

```
10
20
30
```

Assigning Same Value to Multiple Variables

```
a = b = c = 100
print(a)
print(b)
print(c)
```

Output

```
100
100
100
```

3. Data Types in Python

Introduction

In the previous section, we learned that variables are used to store data.

Now the question arises:

What type of data can a variable store?

For example:

```
name = "Krupa"
age = 20
percentage = 85.5
result = True
```

Here:

- "Krupa" is text
- 20 is a number
- 85.5 is a decimal number
- True is a logical value

Since different kinds of information are stored, Python classifies them into different **Data Types**.

What is a Data Type?

Definition

A Data Type specifies the type of value stored in a variable.

Python automatically determines the datatype based on the assigned value.

Python Data Types

Python provides several built-in data types that can be broadly classified into the following categories:

1. Numeric Types
2. Sequence Types

- 3. Mapping Type
- 4. Set Types
- 5. Boolean Type
- 6. Binary Types
- 7. None Type

Numeric Types

Used for storing numerical values.

Integer (int)

Whole numbers without decimal point.

```
a = 10
b = -50

print(type(a))
```

Output:

```
<class 'int'>
```

Float

Numbers containing decimal values.

```
salary = 25000.50

print(type(salary))
```

Output:

```
<class 'float'>
```

Complex

Numbers containing real and imaginary parts.

Syntax:

```
a + bj
```

Example:

```
num = 3 + 4j

print(num)
print(type(num))
```

Output:

```
(3+4j)
<class 'complex'>
```

2. Sequence Types

Sequence means a collection of elements arranged in order.

Python provides:

```
String
List
Tuple
Range
```

String (str)

Collection of characters enclosed in quotes.

```
name = "Krupa"
print(type(name))
```

Output:

```
<class 'str'>
```

List

Ordered collection of items.

Characteristics:

- Ordered
- Mutable (can be modified)
- Allows duplicate values

Example:

```
students = ["Riya", "Priya", "Amit"]
print(students)
print(type(students))
```

Output:

```
['Riya', 'Priya', 'Amit']  
<class 'list'>
```

Tuple

Ordered collection of items.

Characteristics:

- Ordered
- Immutable
- Allows duplicates

Example:

```
marks = (80, 85, 90)  
print(type(marks))
```

Output:

```
<class 'tuple'>
```

Range

Used for generating sequences of numbers.

Example:

```
r = range(1, 6)  
print(type(r))
```

Output:

```
<class 'range'>
```

3. Mapping Type

Python provides one mapping datatype:

Dictionary (dict)

Stores data in key-value pairs.

Example:

```
student = {  
    "name": "Krupa",  
    "age": 20  
}
```

```
print(type(student))
```

Output:

```
<class 'dict'>
```

4. Set Types

Used to store unique values.

Python provides:

```
Set  
Frozen Set
```

Set

Characteristics:

- Unordered
- Mutable
- No duplicate values

Example:

```
numbers = {10, 20, 30, 40}  
  
print(type(numbers))
```

Output:

```
<class 'set'>
```

Frozen Set

Immutable version of set.

Example:

```
fs = frozenset([10, 20, 30])  
  
print(type(fs))
```

Output:

```
<class 'frozenset'>
```

5. Boolean Type

Stores logical values.

Possible values:

```
True  
False
```

Example:

```
result = True  
  
print(type(result))
```

Output:

```
<class 'bool'>
```

6. Binary Types

Used for storing binary data.

Python provides:

```
bytes  
bytearray  
memoryview
```

Bytes

Immutable binary data.

Example:

```
data = b"Python"  
  
print(type(data))
```

Output:

```
<class 'bytes'>
```

Bytearray

Mutable binary data.

Example:

```
data = bytearray(5)

print(type(data))
```

Output:

```
<class 'bytearray'>
```

7. None Type

Represents absence of value.

Example:

```
x = None

print(type(x))
```

Output:

```
<class 'NoneType'>
```

Complete Classification Table

Category	Data Types
Numeric	int, float, complex
Sequence	str, list, tuple, range
Mapping	dict
Set	set, frozenset
Boolean	bool
Binary	bytes, bytearray,
Special	NoneType

Types of Operators in Python

Python provides the following types of operators:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison (Relational) Operators
4. Logical Operators
5. Bitwise Operators
6. Membership Operators
7. Identity Operators

1.Arithmetic Operators

Used to perform mathematical calculations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b
//	Floor Division	a // b
**	Exponentiation	a ** b

Addition (+)

```
a = 10
b = 5

print(a + b)
```

Output:

15

Subtraction (-)

```
print(a - b)
```

Output:

5

Multiplication (*)

```
print(a * b)
```

Output:

50

Division (/)

```
print(a / b)
```

Output:

2.0

Modulus (%)

Returns remainder.

```
print(10 % 3)
```

Output:

1

Floor Division (//)

Returns quotient without decimal part.

```
print(10 // 3)
```

Output:

3

Exponentiation (**)

Used to calculate power.

```
print(2 ** 3)
```

Output:

8

Because:

$$2 \times 2 \times 2 = 8$$

Arithmetic Operator Program Example:

```
a = 20
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Modulus:", a % b)
print("Floor Division:", a // b)
print("Power:", a ** b)
```

2. Assignment Operators in Python

Introduction

In Python, variables are used to store values. To store a value in a variable, we use the **assignment operator (=)**.

Assignment operators are used to assign values to variables and, in some cases, perform an operation and assignment in a single statement.

What is an Assignment Operator?

Definition

An Assignment Operator is used to assign a value to a variable.

Syntax

```
variable = value
```

Example

```
a = 10
```

Here:

- a is a variable
- = is the assignment operator
- 10 is the value assigned to a

Why Do We Need Assignment Operators?

Without assignment operators, variables cannot store values.

Example:

```
name = "Krupa"  
age = 20
```

The values are stored in memory and can be used later in the program.

Types of Assignment Operators in Python

Operator	Description	Example
=	Assign	a = 10
+=	Add and Assign	a += 5
-=	Subtract and Assign	a -= 5
*=	Multiply and Assign	a *= 5
/=	Divide and Assign	a /= 5
%=	Modulus and Assign	a %= 5
//=	Floor Divide and Assign	a //= 5
**=	Power and Assign	a **= 2

1. Simple Assignment Operator (=)

Assigns a value to a variable.

Example

```
a = 10  
  
print(a)
```

Output

```
10
```

2. Add and Assign Operator (+=)

Adds a value and stores the result back in the same variable.

Example

```
a = 10  
  
a += 5  
  
print(a)
```

Output

15

Equivalent Statement

```
a = a + 5
```

3. Subtract and Assign Operator (-=)

Subtracts a value and stores the result back in the variable.

Example

```
a = 20  
a -= 5  
print(a)
```

Output

15

Equivalent Statement

```
a = a - 5
```

4. Multiply and Assign Operator (*=)

Multiplies a value and stores the result.

Example

```
a = 10  
a *= 3  
print(a)
```

Output

30

Equivalent Statement

```
a = a * 3
```

5. Divide and Assign Operator (/=)

Divides a value and stores the result.

Example

```
a = 20  
a /= 4  
print(a)
```

Output

5.0

Equivalent Statement

```
a = a / 4
```

6. Modulus and Assign Operator (%=)

Calculates the remainder and stores it.

Example

```
a = 17  
a %= 5  
print(a)
```

Output

2

Equivalent Statement

```
a = a % 5
```

7. Floor Division and Assign Operator (//=)

Performs floor division and stores the result.

Example

```
a = 17  
a //= 5  
print(a)
```

Output

3

Equivalent Statement

```
a = a // 5
```

8. Power and Assign Operator (**=)

Calculates power and stores the result.

Example

```
a = 2
a **= 3
print(a)
```

Output

8

Equivalent Statement

```
a = a ** 3

a *= 2
print("After *= :", a)
```

3. Comparison (Relational) Operators in Python

Introduction

In our daily life, we compare things regularly.

Examples:

- Is 20 greater than 10?
- Is Rahul's age equal to Riya's age?
- Is the student's mark greater than the passing mark?

Similarly, Python provides **Comparison Operators** to compare two values.

What is a Comparison Operator?

Definition

A Comparison Operator is used to compare two values or variables.

The result of a comparison is always:

True

or

False

Types of Comparison Operators

Operator	Meaning
==	Equal To
!=	Not Equal To
>	Greater Than
<	Less Than
>=	Greater Than or Equal To
<=	Less Than or Equal To

1. Equal To (==)

Checks whether two values are equal.

Example

```
a = 10
b = 10

print(a == b)
```

Output

True

Because both values are equal.

Example

```
a = 10
b = 20

print(a == b)
```

Output

False

2. Not Equal To (!=)

Checks whether two values are different.

Example

```
a = 10
b = 20

print(a != b)
```

Output

True

Because 10 and 20 are not equal.

3. Greater Than (>)

Checks whether left value is greater than right value.

Example

```
a = 20
b = 10

print(a > b)
```

Output

True

4. Less Than (<)

Checks whether left value is less than right value.

Example

```
a = 10
b = 20

print(a < b)
```

Output

True

5. Greater Than or Equal To (>=)

Example

```
marks = 35  
print(marks >= 35)
```

Output

```
True
```

Real-Life Example

Passing criteria:

```
marks = 40  
print(marks >= 35)
```

Output

```
True
```

Student passed.

6. Less Than or Equal To ($\leq$)

Example

```
age = 17  
print(age <= 18)
```

Output

```
True
```

4. Logical Operators in Python

Introduction

Sometimes we need to check multiple conditions together.

Example:

A student passes only if:

- Marks ≥ 35
- Attendance $\geq 75\%$

In such cases, Comparison Operators alone are not enough.

We use **Logical Operators**.

What is a Logical Operator?

Definition

Logical Operators are used to combine multiple conditions and return True or False.

Types of Logical Operators

Operator	Meaning
and	Both conditions must be True
or	At least one condition must be True
not	Reverses the result

1. AND Operator

Rule

Result is True only when both conditions are True.

Truth Table

Condition 1	Condition 2	Result
True	True	True
True	False	False
False	True	False
False	False	False

Example

```
age = 20  
print(age >= 18 and age <= 60)
```

Output

True

Because both conditions are True.

Another Example

```
print(10 > 5 and 20 > 10)
```

Output

True

2. OR Operator

Rule

Result is True if at least one condition is True.

Truth Table

Condition 1	Condition 2	Result
True	True	True
True	False	True
False	True	True
False	False	False

Example

```
print(10 > 5 or 20 < 10)
```

Output

True

Because first condition is True.

Example

```
print(False or False)
```

Output

False

3. NOT Operator

Rule

Reverses the result.

Original	Result
True	False
False	True

Example

```
print(not True)
```

Output

```
False
```

Example

```
print(not False)
```

Output

```
True
```

5. Bitwise Operators

Introduction

Bitwise Operators perform operations on the binary (0 and 1) representation of numbers.

These operators are mainly used in:

- System Programming
- Networking

Common Bitwise Operators

Operator	Description
&	Bitwise AND
^	^
^	Bitwise XOR

Operator	Description
~	Bitwise NOT
<<	Left Shift
>>	Right Shift

Example: Bitwise AND (&)

```
a = 5
b = 3

print(a & b)
```

Output

```
1
Because:
5 = 101
3 = 011
-----
& = 001
which is 1.
```

Example: Bitwise OR (|)

```
print(5 | 3)
Output:
7
```

6. Membership Operators

Introduction

Membership Operators are used to check whether a value exists in a sequence such as:

- String
- List
- Tuple
- Set
- Dictionary

Python provides two membership operators:

Operator	Meaning
in	Value is present
not in	Value is not present

1. in Operator

Checks whether a value exists in a sequence.

Example

```
name = "Python"  
print("P" in name)
```

Output

```
True
```

Because `P` is present in the string `"Python"`.

2. not in Operator

Checks whether a value is not present in a sequence.

Example

```
name = "Python"  
print("Z" not in name)
```

Output

```
True
```

7. Identity Operators

Introduction

Identity Operators are used to check whether two variables refer to the **same object in memory**.

Python provides two identity operators:

Operator	Meaning
is	Returns True if both variables refer to the same object
is not	Returns True if both variables refer to different objects

1. is Operator

Example

```
a = 10
b = 10

print(a is b)
```

Output

True

Both variables refer to the same value.

2. is not Operator

Example

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a is not b)
```

Output

True

Input and Output Functions in Python

Introduction

Every program performs two basic operations:

Input

Taking data from the user.

Output

Displaying information to the user.

Real-Life Example

Consider an ATM machine.

Input

```
Enter ATM PIN
Enter Amount
```

The ATM receives information from the user.

Output

```
Transaction Successful
Remaining Balance: ₹5000
```

The ATM displays information to the user.

Similarly, in Python:

```
Input → Processing → Output
```

Example:

```
Enter Number → Add 10 → Display Result
```

Output Function: print()

What is print()?

The `print()` function is used to display output on the screen.

Syntax

```
print(value)
```

Example 1

```
print("Hello World")
```

Output

```
Hello World
```

Example 2

```
print("Welcome to Python")
```

Output

```
Welcome to Python
```

Printing Numbers

```
print(100)
```

Output

```
100
```

Printing Variables

```
name = "Krupa"  
print(name)
```

Output

```
Krupa
```

Printing Multiple Values

```
name = "Krupa"  
age = 20  
  
print(name, age)
```

Output

```
Krupa 20
```

Using Labels

```
name = "Krupa"  
age = 20  
  
print("Name:", name)  
print("Age:", age)
```

Output

```
Name: Krupa  
Age: 20
```

This makes output more readable.

Separator (sep)

By default, Python separates values with a space.

Example:

```
print("Python", "Java", "C")
```

Output:

```
Python Java C
```

We can change the separator.

```
print("Python", "Java", "C", sep="-")
```

Output:

```
Python-Java-C
```

End Parameter

By default, `print()` moves to the next line.

Example:

```
print("Hello")  
print("Students")
```

Output:

```
Hello  
Students
```

Using `end`:

```
print("Hello", end=" ")  
print("Students")
```

Output:

```
Hello Students
```

Input Function: `input()`

What is `input()`?

The `input()` function is used to accept data from the user.

Syntax

```
input("message")
```

Example 1

```
name = input("Enter your name: ")  
print(name)
```

Sample Run

```
Enter your name: Krupa  
Krupa
```

Example 2

```
city = input("Enter your city: ")  
print("City:", city)
```

Sample Run

```
Enter your city: Anand  
City: Anand
```

Understanding How input() Works

Consider:

```
name = input("Enter your name: ")
```

Step 1

Python displays:

```
Enter your name:
```

Step 2

User enters:

```
Krupa
```

Step 3

Python stores:

```
Krupa
```

inside variable `name`.

Step 4

We can use it later.

```
print(name)
```

Output:

Krupa

Important Note

`input()` always returns data as a String.

Example:

```
num = input("Enter a number: ")  
print(type(num))
```

Input:

10

Output:

```
<class 'str'>
```

Even though the user entered a number, Python stores it as a string.

Why Type Conversion is Needed

Example:

```
num1 = input("Enter First Number: ")  
num2 = input("Enter Second Number: ")  
  
print(num1 + num2)
```

Input:

10
20

Output:

1020

Because Python joins two strings.

Correct Program

```
num1 = int(input("Enter First Number: "))  
num2 = int(input("Enter Second Number: "))  
  
print(num1 + num2)
```